

2025

BENGALURU
CITY UNIVERSITY

BY PASHA



[JAVA LAB MANUAL]

Program 1: . Java Program to Display "Hello World" and Display the Size of All Data Types.

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        // Display "Hello World"  
  
        System.out.println("Hello World");  
  
        // Display the size of all primitive data types  
  
        System.out.println("Size of byte: " + Byte.SIZE + " bits");  
  
        System.out.println("Size of short: " + Short.SIZE + " bits");  
  
        System.out.println("Size of int: " + Integer.SIZE + " bits");  
  
        System.out.println("Size of long: " + Long.SIZE + " bits");  
  
        System.out.println("Size of float: " + Float.SIZE + " bits");  
  
        System.out.println("Size of double: " + Double.SIZE + " bits");  
  
        System.out.println("Size of char: " + Character.SIZE + " bits");  
  
    }  
}
```

OUTPUT:

Hello World

Size of byte: 1 bytes

Size of short: 2 bytes

Size of int: 4 bytes

Size of long: 8 bytes

Size of float: 4 bytes

Size of double: 8 bytes

Size of char: 2 bytes

Explanation:

- **public class HelloWorld:** Defines a public class named HelloWorld.
- **public static void main(String[] args):** The main method where the program execution begins.
- **System.out.println("Hello World");** Prints "Hello World" to the console.
- **Byte.SIZE, Short.SIZE, etc.:** These are constants that return the size of the respective data types in bits.
- **System.out.println(...):** Prints the size of each data type to the console.

2. Java Program to Implement the Usage of Static, Local, and Global Variables.

```
public class VariableExample {  
  
    // Global variable static int staticVar = 10; // Static variable  
  
    int instanceVar = 20; // Instance variable (global to the instance)  
  
  
    public void display() {  
  
        // Local variable  
  
        int localVar = 30; // Local variable  
  
        System.out.println("Static Variable: " + staticVar);  
  
        System.out.println("Instance Variable: " + instanceVar);  
  
        System.out.println("Local Variable: " + localVar);  
    }  
  
    public static void main(String[] args) {  
  
        VariableExample example = new VariableExample();  
  
        example.display();  
    }  
}
```

```
}
```

```
}
```

OUTPUT:

Static Variable: 10

Global Variable: 20

Local Variable: 30

Explanation:

- **public class VariableExample:** Defines a public class named VariableExample.
- **static int staticVar = 10;** A static variable shared among all instances of the class.
- **int instanceVar = 20;** An instance variable unique to each instance of the class.
- **public void display():** A method to display the values of the variables.
- **int localVar = 30;** A local variable defined within the display method.
- **System.out.println(...):** Prints the values of the static, instance, and local variables.
- **VariableExample example = new VariableExample();** Creates an instance of VariableExample.
- **example.display();** Calls the display method to show the variable values.

3. Java Program to Implement String Operations: Length, Concatenate, Substring.

```
public class StringOperations {  
  
    public static void main(String[] args) {  
  
        String str1 = "Hello";  
  
        String str2 = "World";  
  
        // String length
```

```
System.out.println("Length of str1: " + str1.length());
```

```
// String concatenation
```

```
String concatenated = str1 + " " + str2;
```

```
System.out.println("Concatenated String: " + concatenated); // Substring
```

```
String substring = str1.substring(1, 4); // Extracts characters from index 1  
to 3
```

```
System.out.println("Substring of str1: " + substring);
```

```
}
```

```
}
```

OUTPUT:

Length of str1: 5

Concatenated String: Hello World

Substring (0 to 4): Hell

Explanation:

- **public class StringOperations:** Defines a public class named StringOperations.
- **String str1 = "Hello";** Initializes a string variable str1.
- **String str2 = "World";** Initializes another string variable str2.
- **str1.length():** Returns the length of str1.
- **String concatenated = str1 + " " + str2;** Concatenates str1 and str2 with a space in between.
- **str1.substring(1, 4):** Extracts a substring from str1 starting at index 1 and ending at index 3.
- **System.out.println(...):** Prints the length, concatenated string, and substring to the console.

4. Java Program to Find the Maximum of Three Numbers

```
import java.util.Scanner;
```

```
public class MaxOfThree {  
  
    public static void main(String[] args) {  
  
        Scanner scanner = new Scanner(System.in);  
  
        // Input three numbers  
  
        System.out.print("Enter first number: ");  
  
        int num1 = scanner.nextInt();  
  
        System.out.print("Enter second number: ");  
  
        int num2 = scanner.nextInt();  
  
        System.out.print("Enter third number: ");  
  
        int num3 = scanner.nextInt();  
  
        // Find the maximum number int max = num1; // Assume num1 is the maximum initially  
  
        if (num2 > max) {  
            max = num2; // Update max if num2 is greater  
        }  
  
        if (num3 > max) {  
            max = num3; // Update max if num3 is greater  
        }  
  
        // Display the maximum number  
  
        System.out.println("The maximum number is: " + max);  
  
        // Close the scanner  
  
        scanner.close();  
  
        }  
  
    }
```

INPUT:

Enter first number: 10

Enter second number: 25

Enter third number: 15

OUTPUT:

Maximum number is: 25

Explanation:

- **import java.util.Scanner;;** Imports the Scanner class for taking input from the user.
- **public class MaxOfThree:** Defines a public class named MaxOfThree.
- **public static void main(String[] args):** The main method where the program execution begins.
- **Scanner scanner = new Scanner(System.in);** Creates a Scanner object to read input from the console.
- **System.out.print(...):** Prompts the user to enter three numbers.
- **int num1 = scanner.nextInt();** Reads the first number from the user.
- **int num2 = scanner.nextInt();** Reads the second number from the user.
- **int num3 = scanner.nextInt();** Reads the third number from the user.
- **int max = num1;** Initializes max with the value of num1.
- **if (num2 > max) { max = num2; }** Checks if num2 is greater than max, and if so, updates max.
- **if (num3 > max) { max = num3; }** Checks if num3 is greater than max, and if so, updates max.
- **System.out.println("The maximum number is: " + max);** Prints the maximum number to the console
- **scanner.close();** Closes the scanner to prevent resource leaks.

5. Java Program to Check Whether a Number is Odd or Even.

```
import java.util.Scanner;
```

```
public class OddEvenCheck {  
  
    public static void main(String[] args) {  
  
        Scanner scanner = new Scanner(System.in);  
  
        // Input a number System.out.print("Enter a number: ");  
  
        int number = scanner.nextInt();  
  
        // Check if the number is odd or even  
  
        if (number % 2 == 0) {  
  
            System.out.println(number + " is even.");  
  
        } else {  
  
            System.out.println(number + " is odd.");  
  
        }  
  
        // Close the scanner  
  
        scanner.close();  
  
        }  
  
    }
```

INPUT:

Enter a number: 7

OUTPUT:

7 is Odd.

Explanation:

- **import java.util.Scanner;** Imports the Scanner class for taking input from the user.
- **public class OddEvenCheck:** Defines a public class named OddEvenCheck.
- **public static void main(String[] args):** The main method where the program execution begins.

- **Scanner scanner = new Scanner(System.in);** Creates a Scanner object to read input from the console.
- **System.out.print("Enter a number: ");** Prompts the user to enter a number.
- **int number = scanner.nextInt();** Reads the number from the user.
- **if (number % 2 == 0):** Checks if the number is divisible by 2 (even).
- **System.out.println(number + " is even.");** Prints that the number is even if the condition is true.
- **else:** If the number is not even, it must be odd.
- **System.out.println(number + " is odd.");** Prints that the number is odd.
- **scanner.close();** Closes the scanner to prevent resource leaks.

6. Java Program to Implement Default and Parameterized Constructors

```
class ConstructorDemo {  
    int num;  
    String name;  
    // Default Constructor  
    ConstructorDemo() {  
        num = 0; name = "Default";  
        System.out.println("Default Constructor Called");  
    }  
    // Parameterized Constructor  
    ConstructorDemo(int n, String nm) {  
        num = n;  
        name = nm;  
        System.out.println("Parameterized Constructor Called");  
    }  
}
```

```

    void display() {

        System.out.println("Number: " + num + ", Name: " + name);

    }

    public static void main(String[] args) {

        ConstructorDemo obj1 = new ConstructorDemo(); // Calls default
        constructor

        obj1.display();

        ConstructorDemo obj2 = new ConstructorDemo(10, "John");

        // Calls parameterized constructor

        obj2.display();

    }

}

```

Explanation

- 1. A default constructor initializes num to 0 and name to "Default".**
- 2. A parameterized constructor takes int n and String nm to initialize values.**
- 3. display() prints the values.**
- 4. In main(), we create two objects, one using the default constructor and another using the parameterized constructor.**

Output:

Default Constructor Called

Number: 0, Name: Default

Parameterized Constructor Called

Number: 10, Name: John

7. Java Program to Implement an Array of Objects

```

class Student {

    String name;

```

```
    int age;

    Student(String n, int a) {
        name = n;
        age = a;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    public static void main(String[] args) {
        Student[] students = new Student[3]; // Creating an array of objects
        students[0] = new Student("Alice", 20);
        students[1] = new Student("Bob", 22);
        students[2] = new Student("Charlie", 21);
        for (int i = 0; i < students.length; i++) {
            students[i].display();
        }
    }
}
```

Explanation

- 1. The Student class has name and age attributes.**
- 2. Constructor initializes values.**
- 3. We create an array of Student objects.**
- 4. Each student object is initialized separately.**
- 5. A loop prints the details of all students.**

Output

Name: Alice, Age: 20

Name: Bob, Age: 22

Name: Charlie, Age: 21

8. Java Program to Implement Single Inheritance

```
class Parent {  
    void show() {  
        System.out.println("This is Parent class");  
    }  
}  
  
class Child extends Parent {  
    void display() {  
        System.out.println("This is Child class");  
    }  
}  
  
public class SingleInheritance {  
    public static void main(String[] args) {  
        Child obj = new Child();  
        obj.show(); // Inherited from Parent  
        obj.display(); // Defined in Child  
    }  
}
```

Explanation

- 1. The Parent class has a show() method.**
- 2. The Child class inherits from Parent using extends.**
- 3. The Child class adds a new method display().**

4. In main(), an object of Child can call both methods.

Output

This is Parent class

This is Child class

9. Java Program to Implement Multiple Inheritance Using Interface

```
interface A {  
    void methodA();  
}  
  
interface B {  
    void methodB();  
}  
  
class C implements A, B {  
    public void methodA() {  
        System.out.println("Method from Interface A");  
    }  
    public void methodB() {  
        System.out.println("Method from Interface B");  
    }  
}  
  
public class MultipleInheritance {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.methodA();  
        obj.methodB();  
    }  
}
```

```
}  
  
}
```

Explanation

- 1. Java does not support multiple inheritance with classes but allows it with interfaces.**
- 2. A and B are interfaces.**
- 3. C implements both interfaces and provides method definitions.**
- 4. In main(), an object of C calls both methods.**

Output: Method from Interface A

Method from Interface B

10. Java Program to Implement the Life Cycle of an Applet

```
import java.applet.Applet;  
  
import java.awt.Graphics;  
  
public class AppletLifeCycle extends Applet {  
  
    public void init() {  
  
        System.out.println("Applet Initialized");  
  
    }  
  
    public void start() {  
  
        System.out.println("Applet Started");  
  
    }  
  
    public void paint(Graphics g) {  
  
        System.out.println("Painting Applet");  
  
        g.drawString("Hello Applet", 20, 20);  
  
    }  
  
    public void stop() {
```

```
        System.out.println("Applet Stopped");
    }

    public void destroy() {
        System.out.println("Applet Destroyed");
    }
}
```

Explanation

1. **init()** → Called when the applet is first loaded.
2. **start()** → Called when the applet is started or restarted.
3. **paint()** → Handles drawing on the applet window.
4. **stop()** → Called when the applet is paused.
5. **destroy()** → Called when the applet is closed.

How to Run

Since applets are deprecated in Java, you need to use an older JDK or an applet viewer:

```
<applet code="AppletLifeCycle.class" width="300" height="300">
</applet>
```

Output (Console)

Applet Initialized

Applet Started

Painting Applet

Output (Applet Window)

A window displaying "Hello Applet".

Summary of Programs

***Program**

Default and Parameterized Constructors

Array of Objects

Single Inheritance

Multiple Inheritance

Applet Life Cycle

***Concept used**

Class ,constructor overloading

Array,object-oriented programming

Class inheritance(extends)

Interfaces(implements)

Applets methods(init(),start(),paint(),stop(),destroy())

Program 11: Demonstration of Division by Zero Exception

```
public class DivisionByZeroDemo {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 0;  
        try {  
            int result = a / b;  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Exception caught: Division by zero is not allowed.");  
        }  
    }  
}
```

Explanation:

- **int a = 10; and int b = 0;: Initializes two integers.**
- **Division by zero (a / b) will raise an ArithmeticException.**
- **The try block attempts risky code.**
- **The catch block handles the exception and prints a user-friendly message.**

Input/Output: No input required.

Exception caught: Division by zero is not allowed.

Program 12: Method Overloading for Adding Integers and Float Numbers

```
public class MethodOverloadingSum {
    public static int add(int a, int b) {
        return a + b;
    }

    public static float add(float a, float b) {
        return a + b; } public static int add() {
        int a = 5, b = 10;
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println("Sum of two integers: " + add(4, 6));
        System.out.println("Sum of two floats: " + add(4.5f, 3.5f));
        System.out.println("Default sum: " + add());
    }
}
```

Explanation:

- **Three methods with the same name add() but different parameter lists.**

- **Demonstrates method overloading in Java.**
- **Default method add() is called when no arguments are supplied.**

Output:

Sum of two integers: 10

Sum of two floats: 8.0

Default sum: 15

Program 13: Demonstration of Run-Time Polymorphism

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() { System.out.println("Cat meows");  
    }  
}  
  
public class RuntimePolymorphismDemo {  
    public static void main(String[] args) {  
        Animal a;  
        a = new Dog();  
    }  
}
```

```
        a.sound();  
  
        a = new Cat();  
  
        a.sound();  
    }  
}
```

Explanation:

Base class Animal has method sound().

Subclasses Dog and Cat override the method.

At runtime, Java decides which method to execute based on object type.

Output:

Dog barks

Cat meows

Program 14: Catching Negative Array Size Exception

```
public class NegativeArraySizeDemo {  
    public static void main(String[] args) {  
        try {  
            int size = -5;  
            int[] arr = new int[size];  
        } catch (NegativeArraySizeException e) {  
            System.out.println("Exception caught: Array size cannot be negative.");  
        }  
    }  
}
```

Explanation:

- **Attempting to declare array with negative size throws**

NegativeArraySizeException.

- **Caught and handled using try-catch block.**

Output:

Exception caught: Array size cannot be negative.

Program 15: Null Pointer Exception with finally Block

```
public class NullPointerDemo {  
    public static void main(String[] args) {  
        String str = null;  
        try {  
            System.out.println(str.length());  
        } catch (NullPointerException e) {  
            System.out.println("Exception caught: Cannot access length of  
null string.");  
        } finally {  
            System.out.println("Finally block executed: End of program.");  
        }  
    }  
}
```

Explanation:

- **Accessing length() of a null string causes NullPointerException.**
- **finally block executes irrespective of exception being caught.**

Output:

Exception caught: Cannot access length of null string.

Finally block executed: End of program.

Program 16: Demonstrating the Use of 'super' Keyword

```
class Parent {  
int num = 100;  
void display() {  
    System.out.println("Parent display method");  
}  
}  
  
class Child extends Parent {  
int num = 200;  
void display() {  
    super.display();  
System.out.println("Child display method");  
    System.out.println("Parent num: " + super.num);  
    System.out.println("Child num: " + num);  
}  
}  
  
public class SuperKeywordDemo {  
    public static void main(String[] args){  
        Child c = new Child();  
        c.display();  
    }  
}
```

Explanation:

- **super** is used to call the parent class method and access the parent class variable.
- **Demonstrates method and variable overriding.**

Output:

Parent display method

Child display method

Parent num: 100

Child num: 200

Program 17: Demonstrating the Use of Final Keyword

```
class FinalExample {  
    final int CONSTANT = 10;  
  
    void display() {  
        System.out.println("Value of final variable: " + CONSTANT);  
        // Uncommenting the below line will cause a compile-time error  
        // CONSTANT = 20;  
    }  
}  
  
public class FinalKeywordDemo {  
    public static void main(String[] args) {  
        FinalExample obj = new FinalExample();  
        obj.display();  
    }  
}
```

Explanation:

- **final keyword makes a variable constant (read-only).**
- **Attempt to modify a final variable results in a compile-time error.**

Output:

Value of final variable: 10

Program 18: Demonstration of Constructor Overloading

```
class Student {  
String name;  
int age;  
Student() {  
    name = "Unknown";  
    age = 0;  
}  
Student(String n, int a) {  
    name = n;  
    age = a;  
}  
    void display() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}  
  
public class ConstructorOverloadingDemo {  
public static void main(String[] args) {  
    Student s1 = new Student();  
    Student s2 = new Student("John", 20);  
    s1.display();  
    s2.display();  
}  
}
```

Explanation:

- **Two constructors are defined: default and parameterized.**

- **Demonstrates constructor overloading.**

Output:

Name: Unknown, Age: 0

Name: John, Age: 20

Program 19: Demonstration of Multilevel Inheritance

```
class Grandparent {  
    void show() {  
        System.out.println("Grandparent class");  
    }  
}  
  
class Parent extends Grandparent {  
    void show() {  
        super.show();  
        System.out.println("Parent class");  
    }  
}  
  
class Child extends Parent {  
    void show() {  
        super.show();  
        System.out.println("Child class");  
    }  
}  
  
public class MultilevelInheritanceDemo {  
    public static void main(String[] args) {  
        Child obj = new Child();  
    }  
}
```



```
        obj.show();
    }
}
```

Explanation:

- **Demonstrates inheritance from Grandparent → Parent → Child.**
- **super calls the immediate parent class method.**

Output:

Grandparent class

Parent class

Child class

Program 20: Creating and Accessing Packages

Step 1: Save the following code in a file named

mypack/Message.jav

```
a package mypack;

public class Message {
    public void show() {
        System.out.println("Hello from the package!");
    }
}
```

Step 2: Save this code in another file in the main directory

```
import mypack.Message;

public class PackageDemo {
    public static void main(String[] args) {
        Message msg = new Message();
        msg.show();
    }
}
```

```
}
```

```
}
```

Explanation:

- **Package mypack contains a class Message.**
- **Another class imports and uses the package class.**

Output:

Hello from the package!

